

AWS · CONTAINERS E MICROSERVIÇOS

Containers e Microserviços na AWS

Docker, Amazon ECR, ECS, Fargate, Load Balancing, EKS e CloudFormation, como cada peça funciona e o que a integração entre elas viabiliza.

Docker

ECR

ECS

Fargate

ALB

EKS

CloudFormation

Containers e Microserviços · Atividade prática com demonstração

Roteiro de hoje

- | | |
|---|---|
| 01 Docker — imagem, Dockerfile, layers, portas, volumes | 02 Amazon ECR — repositório de imagens gerenciado |
| 03 Amazon ECS — cluster, task, service, deployment | 04 AWS Fargate — containers sem servidor |
| 05 Elastic Load Balancing — distribuindo tráfego | 06 Amazon EKS — Kubernetes gerenciado (opcional) |
| 07 AWS CloudFormation — tudo isso como código | 08 Demonstração integrada — os sete serviços juntos |

De monólitos a microserviços

Monólito clássico

Uma aplicação única, um deploy único. Escalar significa duplicar tudo, mesmo a parte que não precisa.

Ambientes divergem: "na minha máquina funciona".

Microserviços em containers

Cada serviço empacotado com suas dependências, escalado e implantado de forma independente.

O mesmo container roda igual no notebook e na nuvem.

É exatamente isso que vamos construir: dois microserviços (**web** e **api**), cada um em seu container, rodando na AWS com escala, balanceamento e deploy sem downtime.

PARTE 1

Docker

Empacotando os microsserviços em imagens: o alicerce de tudo que vem depois.

Imagem vs. container

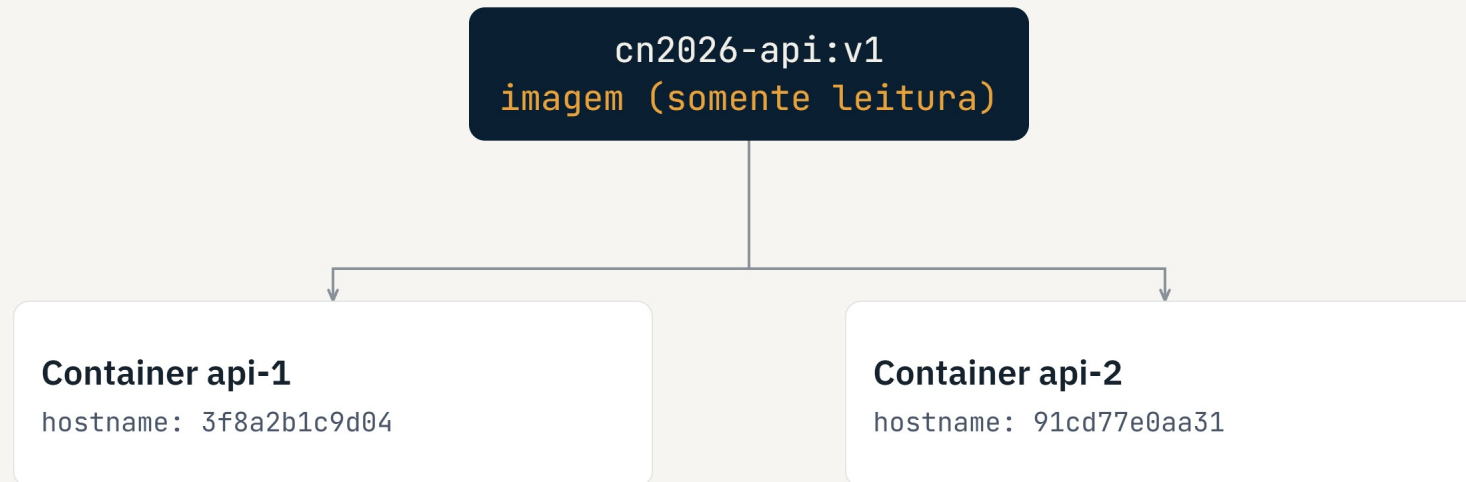


Imagem é o molde: gerada por docker build, não muda depois de pronta.

Container é uma instância em execução dessa imagem, criada por docker run, cada um com seu próprio processo e hostname.

Dockerfile e layers

```
FROM node:20-alpine
WORKDIR /app
COPY package.json .
RUN npm install --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Cada instrução vira uma camada (layer) — imutável e cacheável.

Ordem importa: copiar o package.json antes do resto mantém o npm install em cache enquanto as dependências não mudam.

Rebuild sem mudar dependências → camadas anteriores reaproveitadas, build em segundos.

Portas e volumes

Portas

`-p 9000:3000`

host:9000 → container:3000

O container escuta sempre na mesma porta interna; é o mapeamento HOST:CONTAINER que decide por onde ele é alcançado de fora.

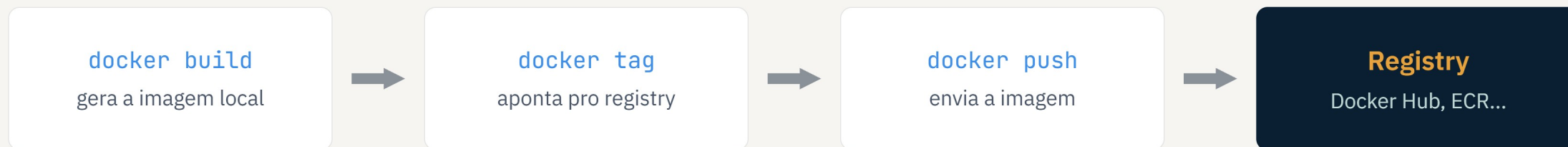
Volumes

`-v dados-cn:/dados`

O sistema de arquivos de um container some junto com ele.

Um **volume** guarda dados fora do ciclo de vida do container, sobrevivem mesmo depois que ele é removido.

Do build ao registry



Um **registry** é um repositório centralizado de imagens. É para lá que enviamos a imagem construída, e de lá que qualquer servidor, cluster ou pipeline a baixa (`docker pull`) para rodar exatamente a mesma versão.

PARTE 2

Amazon ECR

Onde as imagens Docker moram antes de virarem containers rodando na nuvem.

Repositories, imagens e tags

cn2026-web

v1

v2

repositório do microserviço web

cn2026-api

v1

v2

repositório do microserviço api

```
docker tag cn2026-api:v1 SEU_ID.dkr.ecr.us-east-1.amazonaws.com/cn2026-api:v1
docker push SEU_ID.dkr.ecr.us-east-1.amazonaws.com/cn2026-api:v1
```

Cada repositório agrupa as versões de uma imagem por **tag**. Usamos tags de versão (v1, v2) e não latest: só assim o ECS percebe que a imagem mudou.

Lifecycle, scan e integração com o ECS

Scan on push

Toda imagem enviada é escaneada por vulnerabilidades automaticamente.

Lifecycle policy

Mantém só as 5 imagens mais recentes, as antigas expiram sozinhas, sem custo de armazenamento acumulando.

Amazon ECR



Execution Role



Task Fargate rodando

A task definition guarda a URI da imagem; a execution role (a LabRole, no nosso caso) precisa de permissão para puxar do ECR.

PARTE 3

Amazon ECS

O orquestrador que decide quantas cópias de cada container rodar, e onde.

Cluster, task definition e task

01

Cluster

Agrupamento lógico onde os services rodam. Não é uma máquina, é um namespace de orquestração.

02

Task Definition

A "receita": qual imagem do ECR, quanta CPU/memória, quais portas, qual log group. Cada mudança gera uma nova revisão.

03

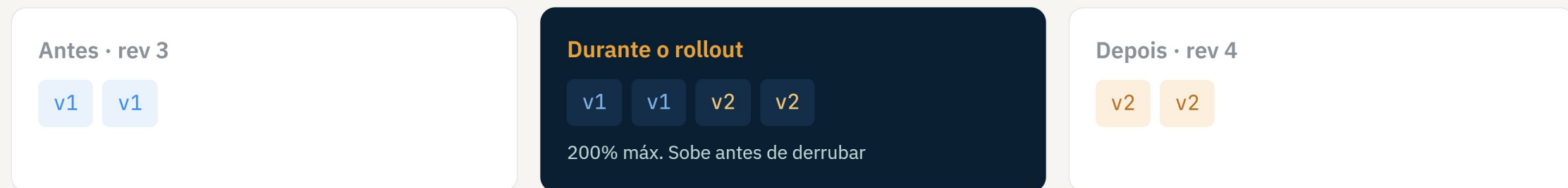
Task

Uma instância em execução da task definition, com seu próprio IP, CPU e memória reservados.

É a mesma relação do Docker: **task definition** está para **task** assim como **imagem** está para **container**, só que orquestrado pelo ECS.

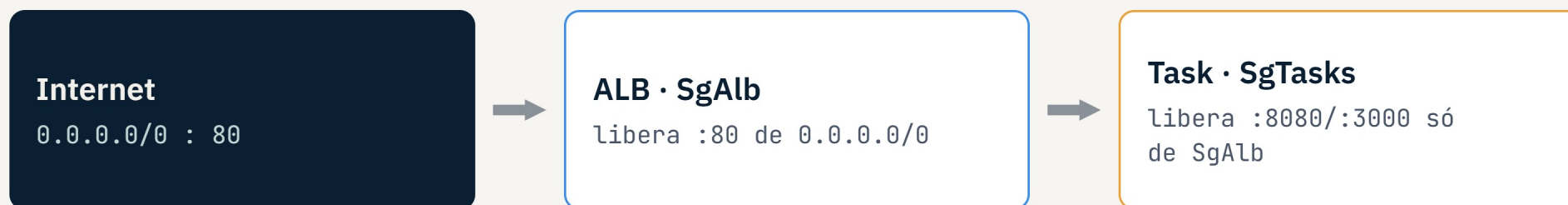
Service, desired count e deployment

O **service** mantém sempre N tasks rodando (o **desired count**), se uma cai, ele cria outra sozinho.



Com `MinimumHealthyPercent:100` e `MaximumPercent:200`, o service nunca fica com menos tasks saudáveis que o desejada, zero downtime no deploy.

Networking: awsvpc e security groups



No modo `awsvpc`, cada task ganha sua própria interface de rede e IP, como se fosse uma mini instância. A única porta de entrada pública é o ALB; as tasks nunca ficam expostas diretamente à internet.

PARTE 4

AWS Fargate

Rodando containers sem provisionar nem gerenciar um único servidor.

Containers sem servidor

O ECS decide **o que** rodar (tasks, services). O Fargate decide **onde** isso roda e a resposta é: você não precisa saber.

Sem Fargate

Você provisiona instâncias EC2, dimensiona o cluster, aplica patches de SO, decide bin-packing das tasks.

Com Fargate

Você declara CPU e memória da task. A AWS provisiona a capacidade, isola e remove ao final — nenhuma instância aparece na sua conta.

Task sizing, isolamento e custo

TaskSize	CPU	Memória	Uso típico
small	256 (.25 vCPU)	512 MB	demo, dev, baixo tráfego
medium	512 (.5 vCPU)	1024 MB	carga moderada
large	1024 (1 vCPU)	2048 MB	carga maior, produção leve

CPU e memória do Fargate só aceitam **combinações fixas**, por isso nosso template CloudFormation usa um único parâmetro TaskSize mapeado com !FindInMap, em vez de deixar escolher CPU e memória soltos.

 **Isolamento:** cada task roda em sua própria micro-VM, não compartilha kernel com tasks de outras contas ou tarefas.

 **Custo:** você paga pelo que a task *reserva*, vCPU e GB por segundo enquanto ela existe, não por uma instância inteira.

Fargate x ECS sobre EC2

Dimensão	Fargate	ECS sobre EC2
Servidores	Não existem na sua conta	Você provisiona e gerencia as instâncias
Patching de SO	A AWS cuida	Você atualiza o SO das instâncias
Escala	Por task, sob demanda	Por instância (Auto Scaling) + por task
Cobrança	vCPU/GB-s reservados pela task	Instância rodando, mesmo ociosa
Isolamento	Forte, microVM por task	Tasks compartilham o SO da instância
Quando usar	Começar rápido, cargas variáveis, sem time de infra dedicado	Controle fino de hardware, GPU, bin-packing em larga escala

PARTE 5

Elastic Load Balancing

A porta de entrada única que distribui o tráfego entre as tasks certas.

Listeners e target groups



O **listener** escuta uma porta e aplica regras. Cada **target group** aponta para um conjunto de tasks, o ALB decide para qual grupo encaminhar cada requisição.

Health checks e roteamento por caminho

Target Group web

GET /health

a cada 10s · 2 OK = healthy · 3 falhas = unhealthy

Target Group api

GET /api/health

a cada 10s · 2 OK = healthy · 3 falhas = unhealthy



Uma task que para de responder ao health check é retirada da rotação automaticamente — o ALB só envia tráfego para quem está **healthy**.

É o path da requisição (/api/*) que decide o target group; o health check decide, dentro dele, quais tasks recebem tráfego.

Distribuindo carga entre as tasks

Target Group api · type: ip

10.0.1.14:3000	healthy
----------------	---------

10.0.2.31:3000	healthy
----------------	---------

Cada task registrada é um alvo pelo seu **IP próprio** (modo awsvpc), por isso o target type é ip, não instance.

```
$ for i in $(seq 12); do curl -s $APP_URL/api/info |  
grep hostname; done | sort | uniq -c
```

```
6 "hostname":"ip-10-0-1-14"
```

```
6 "hostname":"ip-10-0-2-31"
```

12 chamadas na mesma URL, divididas entre as duas tasks — sem o cliente saber disso.

PARTE 6 • OPCIONAL

Amazon EKS

O mesmo problema de orquestração, resolvido com Kubernetes gerenciado.

Os fundamentos do Kubernetes

Control plane

O "cérebro" do cluster, API server, scheduler, etcd. No EKS, a AWS gerencia e cobra por hora.

Nodes

As máquinas que executam os pods, instâncias EC2 num node group, ou um Fargate profile sem instâncias visíveis.

Pods

A menor unidade do Kubernetes, um ou mais containers que compartilham rede e ciclo de vida. É o equivalente à task do ECS.

Services

Um endereço estável de rede sobre um grupo de pods, pode expor um load balancer externo (type: LoadBalancer).

ECS x EKS: mesmo problema, vocabulário diferente

ECS	Kubernetes (EKS)
Cluster	Cluster (control plane + nodes)
Task definition	Pod template (no Deployment)
Task	Pod
Service (desired count)	Deployment (replicas)
Target group + ALB	Service (LoadBalancer) / Ingress
Fargate	Fargate profile ou node group EC2

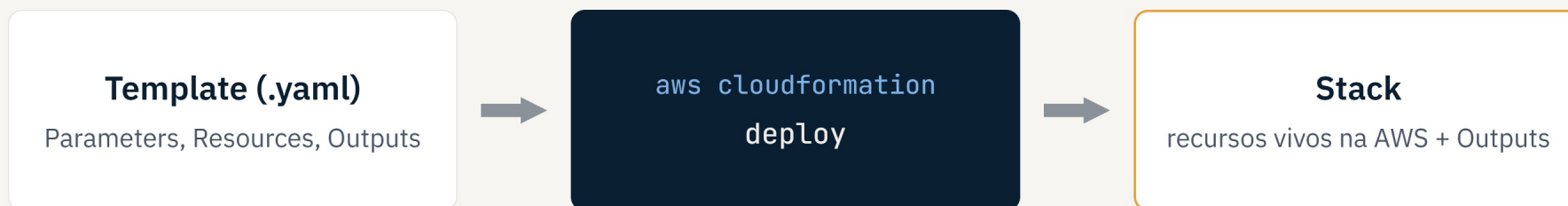
Use **ECS** quando quiser simplicidade e integração nativa com a AWS. Use **EKS** quando precisar do ecossistema Kubernetes (Helm, operators, portabilidade entre nuvens), com o custo extra do control plane por hora.

PARTE 7

AWS CloudFormation

Toda essa infraestrutura descrita em um arquivo de texto, versionável e repetível.

Infraestrutura como código



O mesmo template, aplicado de novo com um parâmetro diferente, **atualiza** a stack em vez de recriar do zero. Apagar a stack remove tudo o que ela criou, na ordem certa, sem checklist manual.

Nossas duas stacks

Parameters

Projeto · VpcId · Subnets
TagWeb · TagApi
QuantidadeTasks · TaskSize

Resources

cn2026-ecr → RepoWeb, RepoApi
cn2026-app → SgAlb, SgTasks, Cluster,
LogGroup, TaskWeb, TaskApi, Alb,
TgWeb, TgApi, Listener, RegraApi,
ServicoWeb, ServicoApi

Outputs

UrlAplicacao
NomeCluster
NomeAlb

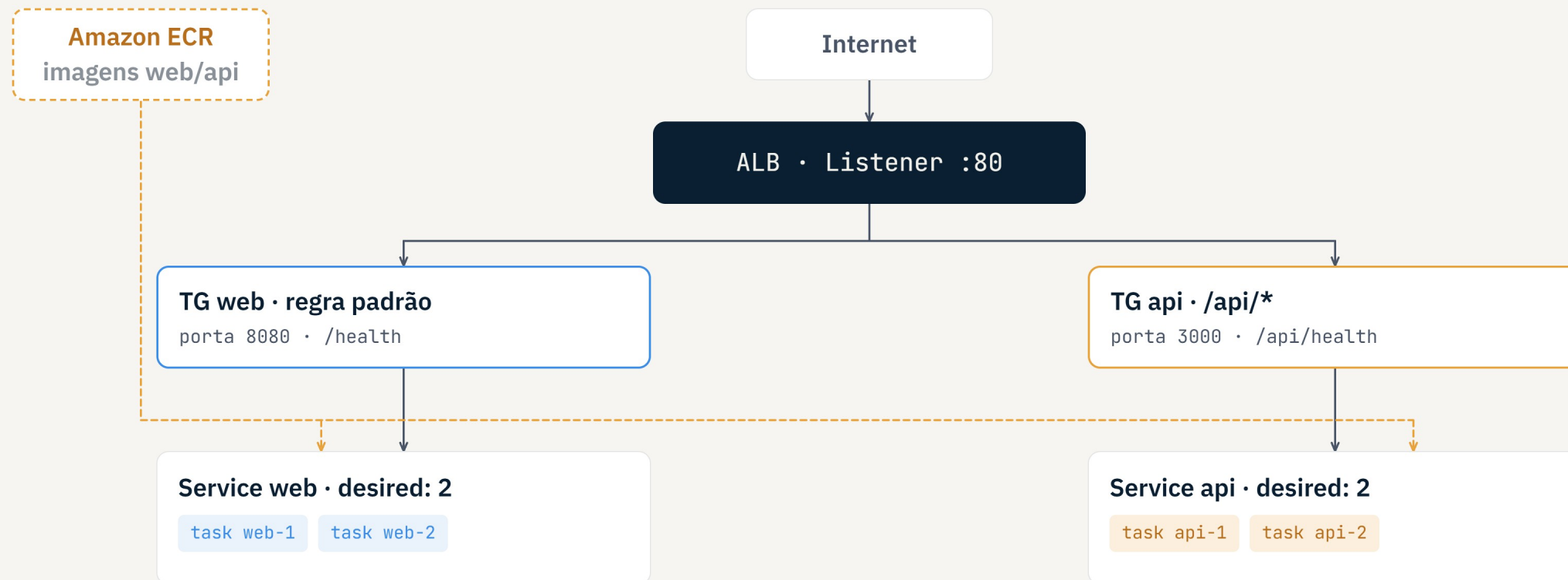
Duas stacks porque têm ciclos de vida diferentes: cn2026-ecr guarda as imagens e sobrevive entre demos; cn2026-app é recriada, escalada e apagada a cada ciclo.

DEMONSTRAÇÃO

Tudo junto, funcionando

Docker, ECR, ECS, Fargate e ALB compondo uma aplicação real, ao vivo.

A arquitetura de ponta a ponta



— tráfego do usuário · - - - imagem publicada no ECR · todos os blocos criados por uma stack CloudFormation

Da imagem ao ALB, em quatro comandos

1

`docker build`

Empacota web e api em
imagens locais

2

`docker push`

Envia para o Amazon ECR (stack
cn2026-ecr)

3

`cloudformation deploy`

Sobe cluster, tasks, services, ALB
(stack cn2026-app)

4

`curl $APP_URL`

Aplicação já respondendo pelo
DNS do ALB

```
$ curl -s $APP_URL/api/info  
{ "servico": "api", "hostname": "ip-10-0-1-14", "versao": "v1", "requisicoes": 1 }
```

Nenhum desses passos exige clicar no console: os quatro comandos acima recriam a aplicação inteira do zero, de forma idêntica, sempre que rodados.

Operar: escalar, atualizar, autocorrigir



Escalar

QuantidadeTasks=4 → deploy_app

O ECS cria as tasks que faltam; o ALB as inclui assim que ficam healthy.



Atualizar sem downtime

TagApi=v2 → deploy_app

Rolling deployment: tasks v2 sobem antes das v1 saírem. Zero erro no meio.



Autocorreção

`aws ecs stop-task` (de propósito)

O service percebe, cria outra task sozinho. O tráfego segue sendo atendido pelas demais.

Em nenhum dos três casos a gente entrou no console clicando: só mudamos um parâmetro e reaplicamos a mesma stack ou deixamos o ECS reagir sozinho.

ENCERRAMENTO

Desligar tudo, na ordem certa



Duas chamadas de `delete-stack` removem services, tasks, ALB, target groups, cluster, security groups, log group e os repositórios do ECR, na ordem inversa da criação, sem custo residual rodando.

ENCERRAMENTO

Em uma frase, cada peça

Docker	empacota a aplicação em imagens portáteis
Amazon ECR	guarda essas imagens com versão e limpeza automática
Amazon ECS	decide quantas tasks rodar e mantém esse número
AWS Fargate	executa as tasks sem um servidor para gerenciar
Load Balancing	distribui tráfego e tira tasks doentes da rotação
Amazon EKS	a mesma orquestração, para quem precisa do Kubernetes
CloudFormation	descreve tudo isso como código, repetível e descartável

Perguntas?

O passo a passo completo, comandos, templates CloudFormation e código das duas aplicações está no guia da atividade, para reproduzir tudo depois da aula.

docs.aws.amazon.com/ecs

docs.aws.amazon.com/cloudformation